

# Thread Scheduling

Scheduling when processes contain multiple threads

## Definition

**Thread scheduling** is the problem of deciding which thread should run when processes may contain multiple threads. The answer depends strongly on whether threads are managed in user space, in the kernel, or both.

## 1. Two Levels of Parallelism

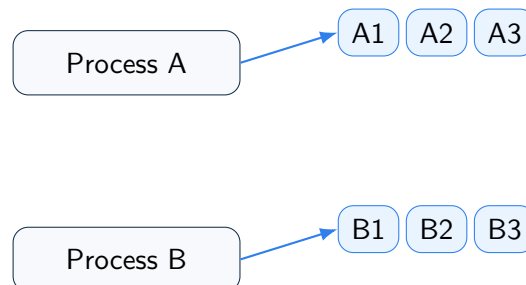
### Processes and Threads

When several processes each have multiple threads, the system has two levels of execution units:

- Processes compete for CPU time.
- Threads inside those processes also compete to run.

### Main Question

Who chooses the next thread: the kernel scheduler or a user-level run-time system inside the process?



## 2. User-Level Threads

### User-Level Threads

With **user-level threads**, the kernel does not know that threads exist. It schedules processes as usual, and a run-time system inside the selected process chooses which user-level thread runs.

### Two Scheduling Decisions

For user-level threads:

1. The kernel chooses a process.
2. The process's run-time system chooses a thread inside that process.

kernel scheduler:

- choose a process, such as process A
- give A one process quantum

run-time system inside A:

- choose one user-level thread, such as A1
- run A1 until it yields, blocks in user space, or A's quantum ends

### 3. No Kernel Clock for User Threads

#### No Thread-Level Preemption

Because the kernel is not aware of user-level threads, it cannot use clock interrupts to preempt one user-level thread and switch to another thread in the same process.

#### Consequence

If thread A1 keeps running, it can consume all of process A's CPU quantum. When A later receives another quantum, A1 may continue again.

#### Limited Damage

This behavior does not steal CPU time from other processes. The kernel still gives other processes whatever share the process scheduler assigns to them.

### 4. User-Level Thread Example

#### 50-ms Process Quantum

Suppose process A has a 50-ms quantum and its threads each do about 5 ms of CPU work before yielding. The run-time system may schedule:

A1 A2 A3 A1 A2 A3 A1 A2 A3 A1

before the kernel switches to process B.

A1 A2 A3 A1 A2 A3 A1 A2 A3 A1 B

Kernel sees A's process quantum, not A's individual threads

#### Not Possible Here

With user-level threads and these parameters, a sequence like A1 B1 A2 B2 A3 B3 is not possible inside A's quantum, because the kernel schedules process A first, not individual A and B threads.

## 5. User-Level Thread Scheduler

### Scheduling Policy

The run-time system inside a process can use familiar scheduling algorithms for its own threads, such as round robin or priority scheduling.

### Cooperative Assumption

The main constraint is that a user-level thread must cooperate by yielding or blocking. There is no kernel timer interrupt forcing it to give way to another user-level thread in the same process.

## 6. Kernel-Level Threads

### Kernel-Level Threads

With **kernel-level threads**, the kernel knows about each thread and schedules individual threads directly.

### Direct Thread Scheduling

The kernel can choose A1, then B1, then A2, even though these threads belong to different processes.

kernel scheduler:

```
choose one runnable kernel-level thread
run it for a thread quantum
preempt it if it exceeds the quantum
```

## 7. Kernel-Level Thread Example

### Interleaving Threads

With a 50-ms quantum and threads that block after 5 ms, the kernel might run:

A1 B1 A2 B2 A3 B3

over a short interval.

A1 B1 A2 B2 A3 B3

Kernel schedules threads directly

### Kernel Choice

The kernel does not have to schedule by process first, although it may consider process ownership when that helps performance or fairness.

## 8. Side-by-Side Comparison

| Feature                  | User-Level Threads                                           | Kernel-Level Threads                                          |
|--------------------------|--------------------------------------------------------------|---------------------------------------------------------------|
| Who knows about threads? | User-level run-time system.                                  | Kernel.                                                       |
| Kernel schedules         | Processes.                                                   | Threads.                                                      |
| Thread preemption        | No kernel preemption among user threads.                     | Kernel can forcibly preempt threads.                          |
| Possible interleaving    | Mostly threads from the selected process during its quantum. | Threads from different processes can interleave directly.     |
| Thread switch cost       | Very low.                                                    | Higher, because the kernel is involved.                       |
| Blocking I/O effect      | One blocking thread may block the whole process.             | One blocking thread does not have to block the whole process. |

## 9. Performance Difference

### User-Level Speed

Switching among user-level threads can take only a small number of machine instructions because it happens inside the process without entering the kernel.

### Kernel-Level Cost

Switching kernel-level threads is more expensive because it involves the kernel. Switching to a thread in another process can require changing the memory map and may disturb the cache.

### Practical Tradeoff

User-level threads are fast to switch, but the kernel cannot schedule them individually. Kernel-level threads cost more to switch, but they give the kernel more control and avoid blocking the whole process on one thread's I/O.

## 10. Same-Process Preference

### Cache and Memory Map

The kernel may know that switching from a thread in process A to another thread in process A is cheaper than switching to a thread in process B.

### Scheduling Hint

If two runnable threads are otherwise equally important, the kernel may prefer a thread from the same process as the thread that just blocked. This can avoid unnecessary memory-map and cache disruption.

## 11. Application-Specific Scheduling

### User-Level Advantage

User-level threads can use an application-specific thread scheduler. The run-time system may understand what each thread does better than the kernel can.

### Web Server

Suppose a Web server has a dispatcher thread and worker threads. If a worker blocks on disk I/O and the dispatcher is ready, the user-level run-time system may choose the dispatcher next so it can start another worker.

### Why This Helps

The application-specific scheduler can maximize useful parallelism because it understands the roles of the threads.

## 12. Kernel Limitation

### Less Application Knowledge

With kernel-level threads, the kernel usually does not know which thread is a dispatcher, which is a worker, or which action would maximize application-level progress.

### Possible Workaround

Threads can be assigned different priorities, but priority numbers may still not capture the full application-specific policy.

## 13. Choosing Between Them

| Need                                                   | Better Fit            |
|--------------------------------------------------------|-----------------------|
| Very fast thread switching                             | User-level threads.   |
| Application-specific scheduling policy                 | User-level threads.   |
| Kernel preemption of individual threads                | Kernel-level threads. |
| One thread blocking without stopping the whole process | Kernel-level threads. |
| Interleaving threads from different processes directly | Kernel-level threads. |

## 14. Big Picture

### Main Conclusion

Thread scheduling depends on where threads are implemented. With user-level threads, the kernel schedules processes and the run-time system schedules threads inside each process. With kernel-level threads, the kernel schedules individual threads directly.

### Exam Shortcut

Remember: user-level thread switching is fast and application-aware, but the kernel cannot preempt or schedule those threads individually. Kernel-level threads are slower to switch, but the kernel can preempt them, interleave threads from different processes, and avoid blocking an entire process when one thread waits for I/O.

## 15. Quick Review

### Key Points

- Multithreaded systems have both process-level and thread-level parallelism.
- With user-level threads, the kernel schedules processes, not threads.
- A user-level run-time system chooses which thread runs inside the selected process.
- User-level threads are not preempted individually by kernel clock interrupts.
- A badly behaved user-level thread can consume its process's whole quantum.
- Other processes are still protected by the kernel's process scheduler.
- With kernel-level threads, the kernel schedules individual threads.
- Kernel-level scheduling can produce sequences such as A1 B1 A2 B2 A3 B3.
- User-level thread switching is much faster.
- Kernel-level threads avoid blocking the whole process when one thread blocks on I/O.
- The kernel may prefer switching to another thread in the same process to reduce overhead.
- User-level thread schedulers can use application-specific knowledge.